WEB DEVELOPMENT

COMPLETE 20-PAGE STUDENT NOTES

HTML • CSS • JavaScript • Bootstrap • HTTP • DOM • Responsive Design • Frontend • Backend • Database • Flask • Security • Deployment

HTML Structure & Content	CSS Design & Layout	JavaScript Logic & Interaction
Frontend Browser-side development	Backend Server-side development	Database Data storage

Designed for diploma / undergraduate students
Concepts + definitions + examples + exam answers + viva revision

2. Introduction to Web Development

Web Development is the process of creating websites and web applications that run through web browsers. It includes designing the user interface, writing client-side and server-side logic, managing data, testing, security and deployment.

Website vs Web Application

Website: Mainly presents information, such as a college information site.
Web application: Provides interactive functionality, such as login, dashboards, online forms and data processing.

Major Areas

- Frontend: What the user sees and interacts with.
- Backend: Server-side processing, business logic and APIs.
- Database: Stores and retrieves application data.
- DevOps/Deployment: Makes the application available to users.

Common Technologies

HTML, CSS, JavaScript, Bootstrap, React, Angular, Node.js, Python/Flask, Django, PHP, Java, MySQL, PostgreSQL and SQLite.

Web Development Flow: Requirement → Design → HTML structure → CSS styling → JavaScript interaction → Backend/API → Database → Testing → Deployment → Maintenance.

3. Client–Server Architecture

The web commonly follows a client–server model. A browser acts as a client and communicates with a web server using network protocols.

Client

The client is usually a browser such as Chrome, Firefox or Edge. It sends requests and displays the response.

Server

The server receives requests, executes server-side logic, accesses databases or other services, and returns a response.

Request–Response Cycle

1. User enters a URL or clicks a link.
2. Browser creates an HTTP request.
3. Server receives the request.
4. Server processes it.
5. Server may query a database.
6. Server returns a response.
7. Browser renders the result.

Static vs Dynamic Web Pages

Static: Content is generally fixed and served as stored files.
Dynamic: Content can be generated based on user input, database data, sessions or application logic.

Important Terms

URL, domain, IP address, DNS, HTTP, HTTPS, browser, server, API, request, response, cookie and session.

Exam Point: Client-side code mainly runs in the browser, while server-side code runs on the server.

4. HTML – Introduction

HTML (HyperText Markup Language) is the standard markup language used to structure content on web pages.

Basic HTML Document

A normal HTML document contains a document type declaration, html element, head section and body section.

```
<!DOCTYPE html>
<html>
  <head><title>My Page</title></head>
  <body>
    <h1>Hello Web</h1>
    <p>Welcome!</p>
  </body>
</html>
```

HTML Elements

An element usually consists of an opening tag, content and a closing tag. Example: Hello .

Attributes

Attributes provide extra information to an element. Example: <img src="photo.jpg" alt="Profile photo">.

Common Tags

<h1>–<h6> headings, paragraph, <a> link, <img> image, / lists, <table> table, <form> form, container.

Remember: HTML gives structure; CSS gives presentation; JavaScript gives behavior.

5. HTML – Text, Links, Images and Lists

Headings and Paragraphs

<h1> is the highest-level heading and <h6> is lower-level. <p> defines a paragraph.

Links

The anchor element creates hyperlinks. The href attribute specifies the destination.

```
<a href="https://example.com">Visit Website</a>
```

Images

The <img> element embeds an image. The alt attribute provides alternative text.

```
<img src="flower.jpg" alt="Flower image" width="300">
```

Lists

Unordered: <ul> with <li> items.
Ordered: <ol> with <li> items.
Description: <dl>, <dt> and <dd>.

Semantic HTML

Semantic tags communicate the meaning/role of content: <header>, <nav>, <main>, <section>, <article>, <aside> and <footer>.

Advantages

Improves accessibility, structure, SEO understanding and maintainability.

6. HTML – Tables and Forms

Tables

HTML tables represent tabular data using rows and cells.

```
<table>
  <tr><th>Name</th><th>Marks</th></tr>
  <tr><td>Asha</td><td>90</td></tr>
</table>
```

Forms

Forms collect user input and send it for processing.

Important Form Controls

<input>, <label>, <textarea>, <select>, <option>, <button>, checkbox, radio button, date and email inputs.

```
<form action="/register" method="post">
  <label>Name</label>
  <input type="text" name="name" required>
  <button type="submit">Register</button>
</form>
```

GET vs POST

GET: Commonly used to retrieve resources; parameters may appear in the URL.
POST: Commonly used to submit data in the request body.

Form Validation

Use appropriate input types and constraints such as required, minlength, maxlength, min, max and pattern.
Server-side validation is still necessary.

7. CSS – Introduction and Syntax

CSS (Cascading Style Sheets) controls the visual presentation and layout of HTML documents.

CSS Syntax

A CSS rule contains a selector and declarations.

```
p { font-size: 18px; margin: 10px; }
```

Ways to Add CSS

1. Inline: style attribute inside an HTML element.
2. Internal: <style> block in the document.
3. External: Separate .css file linked using <link>. External CSS is generally preferred for reusable styling.

Selectors

Element selector: p
Class selector: .card
ID selector: #header
Attribute selector: input[type="text"]
Descendant selector: .card p

Cascade

When multiple rules apply, CSS resolves them based on importance, origin, specificity and source order.

Comments

CSS comments use /* comment */.

Exam Point: Keep structure in HTML and presentation rules in CSS for cleaner and more maintainable projects.

8. CSS – Box Model

The CSS box model describes how the browser calculates the space occupied by an element.

Part	Meaning
Content	Actual text, image or child content
Padding	Space between content and border
Border	Line surrounding padding/content
Margin	Space outside the border

Example

width: 300px; padding: 20px; border: 2px solid; margin: 15px;

box-sizing

content-box is the traditional default. border-box makes the declared width/height include content, padding and border.

Display Property

Common values: block, inline, inline-block, flex, grid, none.

Positioning

static, relative, absolute, fixed and sticky. Positioning should be used according to layout needs rather than as a replacement for responsive layout systems.

Common Problem: Unexpected width/height often comes from padding, border and margin being added under content-box.

9. CSS – Flexbox and Grid

Flexbox

Flexbox is a one-dimensional layout system for arranging items in a row or column.

Important Flex Properties

display: flex; flex-direction; justify-content; align-items; gap; flex-wrap; flex-grow; flex-shrink; order.

```
.container { display:flex; justify-content:space-between; align-items:center; gap:20px; }
```

CSS Grid

Grid is a two-dimensional layout system for rows and columns.

```
.grid { display:grid; grid-template-columns:repeat(3,1fr); gap:20px; }
```

Flexbox vs Grid

Flexbox is usually convenient for one-dimensional alignment and component-level layouts. Grid is powerful for two-dimensional page layouts.

Alignment Concepts

In Flexbox, justify-content controls alignment along the main axis and align-items along the cross axis.

Exam Tip: Learn Flexbox properties with a small diagram showing main axis, cross axis, container and flex items.

10. Responsive Web Design

Responsive Web Design means creating pages that adapt to different screen sizes and devices.

Core Principles

- Flexible layouts
- Relative units such as %, rem, em, vw and vh
- Flexible images
- CSS media queries
- Touch-friendly controls
- Mobile-first thinking

Media Query

A media query applies CSS rules when specified conditions are true.

```
@media (max-width: 768px) { .menu { display:none; } .cards { grid-template-columns:1fr; } }
```

Viewport Meta Tag

For responsive pages, include a suitable viewport declaration in the HTML head so mobile browsers use the intended layout viewport.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Mobile-First Design

Start with a simple layout for small screens and progressively enhance it for larger screens.

Benefits

Better usability, accessibility, maintainability and support for phones, tablets and desktops.

11. JavaScript – Introduction

JavaScript is a programming language widely used to add logic, interaction and dynamic behavior to web pages.

Uses

Form validation, dynamic content, menus, calculations, API calls, browser storage, animations, interactive dashboards and web applications.

Variables

Modern JavaScript commonly uses let and const. var exists for older code but has different scoping behavior.

```
let name = "Yogita";
const age = 18;
let marks = 91;
```

Data Types

String, Number, Boolean, Undefined, Null, BigInt, Symbol and Object.

Operators

Arithmetic: + − * / %
Comparison: ==, ===, !=, !==, <, >, <=, >=
Logical: &&, ||, !
Assignment: =, +=, -=, *=.

Conditionals

if, else if, else and switch.

```
if (marks >= 40) { console.log("Pass"); } else { console.log("Fail"); }
```

12. JavaScript – Functions, Arrays and Objects

Functions

Functions are reusable blocks of code.

```
function add(a, b) { return a + b; }
const result = add(10, 20);
```

Arrow Function

A concise function syntax is commonly used in modern JavaScript.

```
const square = n => n * n;
```

Arrays

Arrays store ordered collections. Common methods include push(), pop(), map(), filter(), find(), forEach() and reduce().

```
const marks = [80, 90, 75];
const doubled = marks.map(x => x * 2);
```

Objects

Objects store properties and methods.

```
const student = { name: "Asha", marks: 90 };
console.log(student.name);
```

Loops

for, while, do...while and for...of are commonly used for repetition.

Scope

let and const are block-scoped. Understanding scope helps prevent accidental variable access and naming conflicts.

13. DOM and Events

DOM (Document Object Model) represents an HTML document as a tree of objects that JavaScript can inspect and modify.

Selecting Elements

document.getElementById("title")
document.querySelector(".card")
document.querySelectorAll("p")

Changing Content

textContent changes text. classList can add/remove/toggle CSS classes. Attributes can be read or changed through suitable DOM methods.

```
const title = document.querySelector("h1");
title.textContent = "Welcome!";
```

Events

Events occur when users or the browser perform actions such as click, submit, input, change, load or keydown.

```
button.addEventListener("click", () => { alert("Hello!"); });
```

Event Handling

Use event listeners to connect user actions to functions. For forms, preventDefault() can stop the browser's default submission when client-side processing is required.

Exam Point: DOM manipulation makes a static HTML page interactive and dynamic.

14. Bootstrap Framework

Bootstrap is a frontend framework that provides reusable CSS and JavaScript components for responsive web interfaces.

Advantages

Responsive grid, utility classes, buttons, forms, cards, navigation bars, modals and many ready-to-use components.

Grid System

Bootstrap layouts are based on rows and columns. Responsive column classes allow different widths at different breakpoints.

Common Components

Navbar, card, alert, button, modal, carousel, accordion, dropdown, form controls and pagination.

Utility Classes

Spacing, display, text alignment, sizing, flex utilities, colors and positioning can often be applied directly through utility classes.

Example

```
<button class="btn btn-primary">Save</button>
<div class="container">...</div>
```

Bootstrap vs Custom CSS: Bootstrap speeds up common UI development; custom CSS gives more precise control over unique designs.

15. HTTP, HTTPS and URLs

HTTP

HyperText Transfer Protocol is an application-layer protocol used for communication between clients and web servers.

Common HTTP Methods

GET: Retrieve data.
POST: Submit/create data.
PUT: Replace/update a resource.
PATCH: Partially update a resource.
DELETE: Delete a resource.

Status Codes

2xx: Success – 200 OK, 201 Created.
3xx: Redirection – 301, 302.
4xx: Client error – 400, 401, 403, 404.
5xx: Server error – 500, 502, 503.

HTTPS

HTTPS is HTTP protected with TLS encryption. It helps protect data in transit and provides server authentication.

URL Structure

A URL can include scheme, domain/host, port, path, query string and fragment.

```
Example: https://example.com/products?id=10#details
```

Important: HTTPS protects communication in transit, but it does not automatically make an application secure.

16. Frontend Development

Frontend development focuses on the part of a web application users see and interact with in a browser.

Core Technologies

HTML → structure
CSS → style/layout
JavaScript → logic/interaction

Frontend Responsibilities

- User interface and navigation
- Form interactions and validation
- Responsive layouts
- Accessibility
- Calling backend APIs
- Showing loading, success and error states

Client-Side Validation

Improves user experience by giving immediate feedback. However, it should never replace server-side validation because users can bypass browser code.

API Communication

JavaScript can communicate with backend services using fetch() or other HTTP clients.

```
fetch('/api/students')
  .then(response => response.json())
  .then(data => console.log(data));
```

Frontend Frameworks

React, Angular and Vue are popular tools for building component-based interfaces.

17. Backend Development

Backend development handles server-side logic, authentication, database operations, APIs, business rules and application integration.

Backend Responsibilities

Routing, request processing, validation, authentication, authorization, database access, file handling, API creation, logging and error handling.

Flask

Flask is a lightweight Python web framework. It can define routes, render templates, receive form data and return responses.

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('index.html')
```

Template Rendering

A server-side template engine can combine HTML with dynamic data before sending the final page to the browser.

API

An API is an interface through which software components communicate. REST-style APIs commonly exchange JSON data over HTTP.

Exam Point: Backend code should validate input and enforce authorization even if the frontend already performs checks.

18. Database and Web Applications

A database stores application data so it can be retrieved, updated and managed efficiently.

Relational Database

Stores data in tables made of rows and columns. Examples include MySQL, PostgreSQL and SQLite.

Important SQL Commands

CREATE – create database objects
INSERT – add records
SELECT – retrieve records
UPDATE – modify records
DELETE – remove records

```
SELECT name, marks FROM students WHERE marks >= 80;
```

Primary Key

Uniquely identifies a row in a table.

Foreign Key

Connects a column to a key in another table, helping represent relationships.

CRUD

Create → INSERT
Read → SELECT
Update → UPDATE
Delete → DELETE

Database Security

Use parameterized queries/ORMs, least privilege, proper authentication, backups and careful validation.

Web App Flow: Browser → Backend route/API → Validation → Database → Response → Browser.

19. Web Security, Testing and Deployment

Common Web Security Risks

XSS: Untrusted input is interpreted as executable browser content.
SQL Injection: Malicious input changes database queries when queries are constructed unsafely.
CSRF: A malicious site tricks an authenticated browser into sending an unwanted request.
Broken access control: Users can access resources they should not.

Basic Protection

Validate input, escape/output-encode untrusted data, use parameterized queries, protect sessions, enforce authorization on the server, use HTTPS, keep dependencies updated and avoid exposing secrets.

Testing

Unit testing checks small components. Integration testing checks interactions. End-to-end testing checks complete user flows. Manual testing helps catch usability and visual problems.

Deployment

Typical steps: prepare application → configure environment variables → install dependencies → configure database → use a production server → enable HTTPS → monitor logs → maintain backups.

> Never hard-code API keys or passwords in public source code.

20. Important Exam Questions + Viva + Quick Revision

Long/Short Answer Questions

1. Define Web Development and explain frontend, backend and database.
2. Explain client–server architecture with request-response cycle.
3. Explain HTML structure and important tags.
4. Explain HTML forms and GET vs POST.
5. Explain CSS selectors and box model.
6. Explain Flexbox and CSS Grid.
7. Explain responsive web design and media queries.
8. Explain JavaScript variables, data types, functions and arrays.
9. Explain DOM and event handling.
10. Explain Bootstrap and its grid system.
11. Explain HTTP methods and status codes.
12. Explain frontend and backend development.
13. Explain Flask routing and templates.
14. Explain database, SQL and CRUD operations.
15. Explain common web security risks and protections.

Viva Questions

- What is HTML?
- What is CSS?
- What is JavaScript?
- What is DOM?
- What is responsive design?
- What is Bootstrap?
- What is HTTP?
- What is HTTPS?
- What is an API?
- What is Flask?
- What is a database?
- What is SQL?
- What is CRUD?
- What is XSS?
- Why should passwords/API keys not be hard-coded?

One-Minute Revision

HTML = Structure | CSS = Design | JS = Behavior | Bootstrap = UI Framework | HTTP = Web Communication | DOM = Page Object Model | Frontend = Browser | Backend = Server | DB = Data Storage | API = Software Communication | CRUD = Create, Read, Update, Delete.

Exam Strategy: For theory answers, write Definition → Working/Steps → Example → Advantages/Limitations → Applications. Draw a simple diagram wherever it improves the answer.